KL 散度与 Loss 函数关系

A Beginner's Note on KL Divergence, Cross Entropy and Log-Likelihood

KL Divergence • Cross Entropy • Log-Likelihood

1 熵（Entropy）的直觉

不确定性就是信息需求

解释：如果一个事件几乎必然发生，我们对它不惊讶；如果它很随机，我们需要更多信息才能描述它。熵度量的就是这种“平均不确定性”。

关键公式

$$H(p) = -\sum_x p(x) \log p(x)$$

这个式子表示：按真实分布 p 发生时，平均需要多少信息量来编码结果。

核心结论：熵是“数据本身的最小平均描述成本”。

逻辑衔接：既然有“最小成本”，下一步自然要问：如果我们用错了分布来编码，会多花多少成本？

2 KL 散度的定义与信息意义

用错分布的额外代价

解释：设 p 是真实世界，q 是模型的相信程度。若数据来自 p，却用 q 设计编码方案，就会产生额外信息开销。

关键公式

$$D_{\mathrm{KL}}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

这个式子表示：平均而言，使用 q 代替 p 会多付出的信息代价。

核心结论： KL 散度衡量的是“相对真实分布的冤枉信息税”。

逻辑衔接：为了把 KL 用到训练里，我们需要引入一个更常见、可直接优化的量：交叉熵。

3 交叉熵与 KL 散度的关系推导

交叉熵是总花费，KL 是额外花费

解释：交叉熵看“总编码长度”，KL 看“比最优编码多花多少”。

关键公式 1

$$H(p,q) = -\sum_x p(x)\log q(x)$$

这个式子表示：真实分布是 p，但编码策略按 q 来做时的平均成本。

关键公式 2

$$H(p,q) = H(p) + D_{\mathrm{KL}}(p\|q)$$

这个式子表示：总花费 = 理想花费 + 额外浪费。

核心结论：当 p 固定时，最小化交叉熵等价于最小化 KL 散度。

逻辑衔接：分类任务里的标签通常是 one-hot，此时交叉熵会进一步化简为 Negative Log Likelihood（负对数似然）形式。

4 分类任务中的 Cross Entropy 与 Negative Log Likelihood

one-hot 标签下 Cross Entropy 的简化形式

解释：在分类任务中，标签通常采用 one-hot 编码：真实类别位置为 1，其余位置为 0。这种特殊结构使得交叉熵的求和会自动消去所有零项，最终只保留”正确类别的概率”这一项。

关键公式 1

$$\mathrm{CrossEntropy} = -\log q(y_{\mathrm{true}})$$

这个式子表示：分类任务中的交叉熵简化为”真实类别概率”的负对数。

关键公式 2

$$\mathrm{NLL}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\log q_\theta(y_i \mid x_i)$$

这个式子表示：负对数似然在数据集层面的平均形式，这正是神经网络训练中最常用的分类损失函数。

核心结论：在 one-hot 编码下，单个样本的交叉熵与该样本的负对数似然在形式上完全相同；对整个数据集求平均后，两者在数值上也等价。

逻辑衔接：现在我们知道了分类损失的数学形式，接下来需要理解这些概率是如何计算的：这通常由 Softmax 函数完成。

5 Softmax + Cross Entropy 与最大似然

logits 到概率，再到可优化目标

解释：神经网络输出 logits（未归一化分数），Softmax 把它转成概率；Cross Entropy 则惩罚正确类别概率太小。

关键公式 1

$$q_\theta(y = k \mid x) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

这个式子表示：Softmax 将第 k 类的 logit z_k 映射为概率。

关键公式 2

$$\mathrm{CE} = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{e^{z_{i,y_i}}}{\sum_j e^{z_{i,j}}}$$

这个式子表示：Softmax + CE 直接对应“最小化正确类负对数概率”。

核心结论： `CrossEntropyLoss(logits, target)` 本质上就是 `LogSoftmax + NLLLoss`。

逻辑衔接：当我们从“每个样本概率尽量大”提升到“全数据概率最大”，就进入 MLE 视角。

举例分类图像的例子

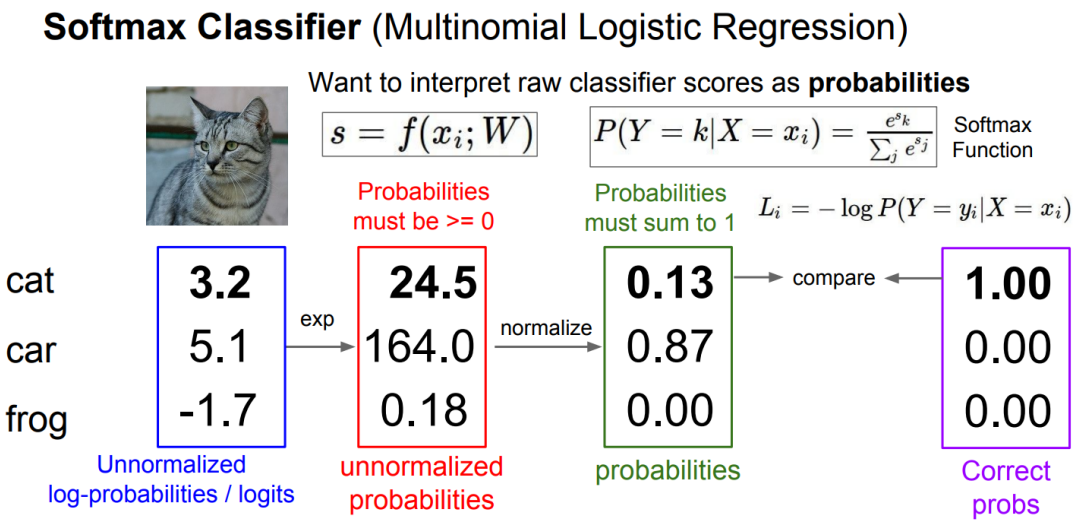

图 1: One-Hot 编码角度

解释：神经网络的输出层产生 logits（原始分数），这些分数尚未归一化，无法直接解释为概率。Softmax 函数的作用是将这些任意范围的分数转换为和为 1 的概率分布，使得我们可以用概率论框架来理解分类问题。

Softmax 的直观理解

考虑一个图像分类任务（如识别猫、车、青蛙），神经网络为每个类别输出一个 logit：

- 猫的 logit：$z_{\text{cat}} = 3.2$
- 车的 logit：$z_{\text{car}} = 5.1$（最高）
- 青蛙的 logit：$z_{\text{frog}} = -1.7$（最低）

这些分数难以解释，因为它们的范围和意义取决于网络的初始化和训练过程。

Softmax 的三个作用：

1. **非线性变换：**通过指数函数 e^{z_k} 放大大的 logit，压小小的 logit
2. **归一化：**除以所有指数之和 $\sum_j e^{z_j}$，确保输出概率和为 1
3. **概率解释：**输出可直接视为”该样本属于各类别的概率”

Softmax 计算示例

对上述 logits 应用 Softmax：

$$\text{未归一化概率} = [e^{3.2}, e^{5.1}, e^{-1.7}] \approx [24.5, 164.0, 0.18]$$

$$\text{归一化因子} = 24.5 + 164.0 + 0.18 = 188.68$$

$$\text{概率分布} = \left[\frac{24.5}{188.68}, \frac{164.0}{188.68}, \frac{0.18}{188.68}\right] \approx [0.13, 0.87, 0.00]$$

最高的 logit（车 5.1）对应最大概率（0.87），最低的 logit（青蛙 -1.7）对应最小概率（0.00）。

核心结论： Softmax 不仅是一个数学函数，更是将任意分数转换为可解释概率的桥梁。在这个过程中，logits 之间的相对大小决定了概率的相对大小，但绝对值本身被”遗忘”了。

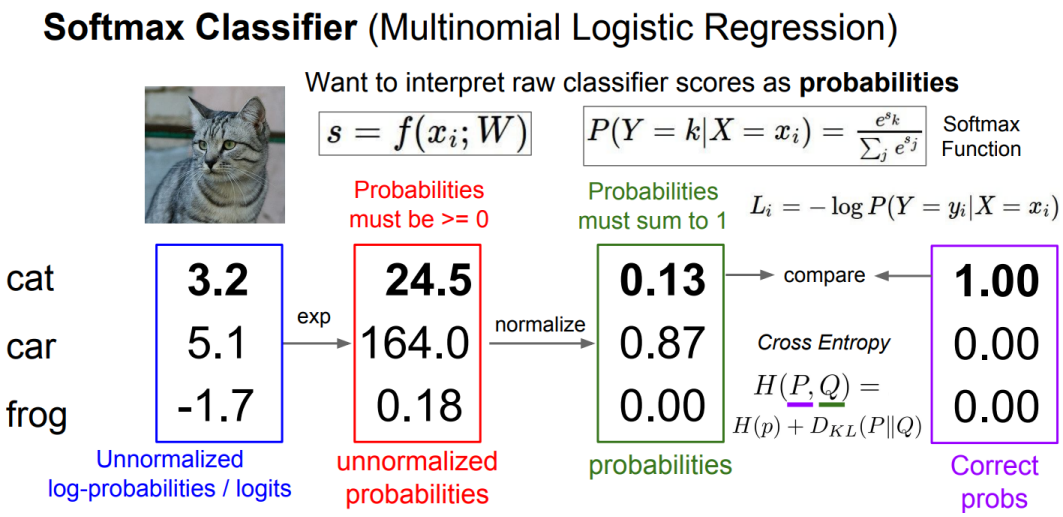

图 2: KL 散度角度

6 MLE 与 KL 散度的统一视角

从参数估计到分布对齐

解释： MLE 说“让观察到的数据最可能出现”；KL 散度说“让模型分布更接近真实分布”。两者是同一个目标的两种语言。

关键公式 1

$$\hat{\theta} = \arg\max_{\theta} \prod_{i=1}^{N} q_\theta(y_i \mid x_i)$$

这个式子表示：最大似然估计直接最大化全数据联合概率。

关键公式 2

$$\hat{\theta} = \arg\min_{\theta} \Big[-\frac{1}{N} \sum_{i=1}^{N} \log q_\theta(y_i \mid x_i) \Big]$$

这个式子表示：取对数并加负号后，MLE 等价于最小化平均 NLL。

核心结论： MLE $\Leftrightarrow$ 最大化对数似然 $\Leftrightarrow$ 最小化 NLL $\Leftrightarrow$（在经验分布意义下）最小化 KL。

逻辑衔接： 统一关系建立后，最后需要避免常见概念混淆，确保在实践中不误用。

7 常见误区澄清

把“像”与“等于”分清楚

解释： 初学者最容易把 Cross Entropy (CE)、Kullback-Leibler divergence (KL)、Negative Log-Likelihood (NLL)、Maximum Likelihood Estimation (MLE) 混成一个词。它们确实紧密相关，但定义对象和语境不同。

常见误区清单：

- KL 不是对称距离：$D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$。
- CE 不总是 KL：严格关系是 $H(p,q) = H(p) + D_{\mathrm{KL}}(p\|q)$。
- CE = NLL，依赖于 one-hot（或等价硬标签）语境。
- `CrossEntropyLoss` 在 PyTorch 中输入应为 logits，而非先 softmax 后概率。

核心结论： 这四个概念“目标一致、表达不同”，但不能在所有条件下机械互换。

逻辑衔接： 理清误区后，就能更稳地理解它们在不同任务中的作用。